INSTRUCTION ARCHITECTURES AND COMPILERS

1. a) How many bits are used to identify a register in MIPS R-type and I-type instructions, and why? [2]

Answer:
5 bits, because there are 32 registers.

b) What types of token are “for”, “fox”, and “0x0” in the C grammar? [3]

Answer:
for is a keyword, fox is an identifier, and 0x0 is a hexadecimal literal.

c) Give an example of a C statement which could map to a single CISC instruction, but maps to more than one MIPS instruction. [4]

Answer:
*The statement “x=*y++;” maps into a single instruction in 68000, but two instructions in MIPS.*

d) Give a MIPS assembly template for compiling C code of the form:

```
while( C ) {
    X;
}
```

[4]

Answer:

```
top_label:
   <Evaluate C and put result in $t0>
   beq $t0, $0, bottom_label
   <Assembly for X>
   beq $0, $0, top_label
bottom_label
```

e) Give an example of a situation where a C compiler can improve the temporal locality of a program. [4]

Answer:
The compiler can keep inner loop variables in registers, in order to avoid loading and storing from the next level (i.e. memory).

f) Give an example of a C arithmetic expression which naïvely maps into two or more MIPS instructions, but could be implemented in one instruction. [4]

Answers

Answer:
*One example would be "x*8", which naively turns into three instructions (load, multiply, mflo), but could be directly implemented as one shift instruction.*

g) Give a regex that is recognised by the Lex tool and matches your CID number, but without using any even decimal digits in the regex. [3]

Answer:
The question as given only requests that it matches the CID, so the simplest solution is to use ".".*

h) MIPS has a branch delay slot, but many modern RISC ISAs do not. State whether you think the delay slot is a good or a bad idea, and then briefly explain your reasoning. [4]

Answer:
Good idea: the delay slot reduces complexity of the processor pipeline, and so potentially reduces power consumption.

Bad idea: the delay slot is often wasted because no instruction fits there, and so reduces IPC.

They can choose good or bad as long as the come up with a valid reason to support it.

i) Give two different ways of encoding a no-op using two different MIPS instructions which *do not* use the zero register. [4]

Answer:
Two examples are "addi $1, $1, 0" and "sll $1,$1, 0"

j) A CPU with k pipeline stages is modified to have $k+1$ stages. Provide an upper bound on the improvement in IPC. [4]

Answer:
There is no reason to think that IPC will get any higher (it will likely be lower), so the upper bound is 1, i.e. the IPC is the same or lower.

k) Write a context-free grammar that generates exactly the language accepted by the regular expression $\mathtt{a}^*\mathtt{b}^*\mathtt{c}^*$. [4]

Answer:
$X ::= ABC, A ::= \varepsilon \mid A\mathtt{a}, B ::= \varepsilon \mid B\mathtt{b}, C ::= \varepsilon \mid C\mathtt{c}$

Answers

2. Imagine you have been asked to extend the MIPS ISA to support n-bit operations, where n is a binary power larger than 32. The register file will still contain 32 registers, and instructions are still 32 bits wide, but each register is now n bits. The purpose is to support large cryptographic operations, so it is anticipated that n might be quite large, for example 1024 or more.

a) Give an example of a MIPS R-type instruction which would not require any modification to its encoding in order to perform the same task in the new ISA. [4]

Answer:
The "and" R-type instruction would still perform the same operation, as it can still identity any three of the 32 registers.

b) Give an example of a MIPS I-type instruction which cannot support the full set of operations over n-bit registers compared to 32-bit registers. [4]

Answer:
The LUI instruction can no longer load half the word, as there are only 16-bits of immediate available.

c) If the ALU uses a two-level carry-select adder, what is the asymptotic critical path of the adder? [4]

Answer:
If the carry-select adder uses a finite number of levels then the critical path is going to be O(n/k) where k is constant. So the carry-select adder is still O(n).

d) The cost of a hardware multiplier has been deemed too expensive, so the designers have instead added two new instructions which allow multiplication to be perfomed in software:

MULX : Multiplies one n-bit register with the bottom 32-bits of another register, producing an n-bit result.

SHIFTX : Shifts a n bit register by $32i$ bits. The shift is left if $i < 0$ or logical right if $i > 0$.

i) Propose an instruction encoding for the MULX instruction using either the R-type or I-type format. [4]

Answer:
For MULX we could simply use the R-type encoding, and use a standard two source and single destination approach. We need to choose register is which, so choose the order `MULX dst, srcFull, srcShort`.

ii) Give MIPS assembly code for full n-bit multiplication using the two new instructions. Assume the two inputs are in registers $a0 and $a1, the output should be placed in $v0, and any other registers can be used as temporaries. [8]

Answer:

```
mv $i, $0
```

Answers

```
    mv $v0, $0
top:
    SHIFTX $t1, $a1, $i // Get limb i of input a1
    MULX $t1, $a0, $t1  // Perform multiplication with input a0
    subu $ni, $0, $i    // Construct negative i
    SHIFTX $t1, $t0, $ni // Shift the product back to the right place
    add $v0, $v0, $t1
    addi $i, $i, 1
    subi, $done, $i, (n/32)
    bnz $done, top
    nop
```

iii) Often cryptographic operations require multiplications on numbers with just pn non-zero bits, where $p << 1$. Propose a new instruction which could take advantage of this property, and estimate the expected time complexity in terms of p and n. [6]

Answer:
If one input is known to be sparse, we could loop over the non-zero words of that input, rather than every single word.

A instruction that could support this would be **LOWESTX** *which takes a single n-bit register, and returns the index of the first non-zero word, or $n/32$ if all words are zero. This would allow the loop to skip over non-zero words, so it becomes O(pn) on average and O(n) worst case.*

Answers

3. Consider the following grammar for programs.

$$
\begin{array}{rcl}
\textit{prog} & ::= & \textit{label}\texttt{:}\ \texttt{IFZERO}\ \textit{var}\ \texttt{GOTO}\ \textit{label}\texttt{;}\ \textit{prog} \\
& | & \textit{label}\texttt{:}\ \texttt{IFNONZERO}\ \textit{var}\ \texttt{GOTO}\ \textit{label}\texttt{;}\ \textit{prog} \\
& | & \textit{label}\texttt{:}\ \textit{var}\ \texttt{:=}\ \textit{expr}\texttt{;}\ \textit{prog} \\
& | & \varepsilon \\
\textit{expr} & ::= & \textit{var} \mid \textit{var}\ \texttt{/}\ \textit{var} \mid \textit{var}\ \texttt{/}\ \textit{literal} \\
\textit{literal} & ::= & \texttt{1} \mid \texttt{2} \mid \ldots \\
\textit{label} & ::= & \texttt{1} \mid \texttt{2} \mid \ldots \\
\textit{var} & ::= & \texttt{a} \mid \texttt{b} \mid \ldots \mid \texttt{z}
\end{array}
$$

All 26 variables are integer-valued and do not need initialising. The variable `a` is designated as the program's input and output. Labels and literals are positive integers and no two statements in a program have the same label. The `/` operator denotes integer division.

a) Devise a program in this grammar that can be improved by common subexpression elimination (CSE). Write down your program both before and after CSE has been applied. Explain why your program is better after transformation. [4]

Answer:
One of many possible programs:

```
1       a := 2 / 2;
2       b := 2 / 2;
```

becomes

```
1       : a := 2 / 2;
2       : b := a
```

This could be better if MOV instructions run quicker than DIVs, as for example in MIPS where two machine instructions are needed.

b) Write down a program in this grammar that could be improved by CSE, but where a naïve analysis of available expressions would not lead to this optimisation opportunity being exploited. Explain the source of this imprecision. Suggest one way the analysis could be made more precise. [6]

Answer:
One possibility:

```
 i)     : a := 1;
ii)     : IFZERO a GOTO 4
iii)    : b := 2 / 2;
iv)     : c := 2 / 2;
```

At line 4, the expression `2 / 2` is actually available, since line 3 will always be executed at runtime. The imprecision comes from the available expression analysis ignoring the values of variables, and hence not realising that the if-condition is always false. The analysis could be made more precise by performing constant propagation first.

c) Devise a program in this grammar that can be made *worse* by CSE. Write down

Answers

your program both before and after CSE has been applied. Explain why your program is worse after transformation. [6]

Answer:

1 : IFZERO a GOTO 4
2 : b := 2 / 2
3 : IFNONZERO a GOTO 5
4 : c := 2 / 2
5 : d := 2 / 2

becomes

1 : e := 2 / 2
2 : IFZERO a GOTO 5
3 : b := e
4 : IFNONZERO a GOTO 6
5 : c := e
6 : d := e

This could be worse because we have increased register pressure by introducing the variable `e`*. It could also be worse because we are now executing more instructions: 1 DIV and 2 MOVs, rather than 2 DIVs – though this depends on whether 1 DIV or 2 MOVs is more expensive.*

Note: this example was more-or-less given in my lecture, but not with this restrictive grammar.

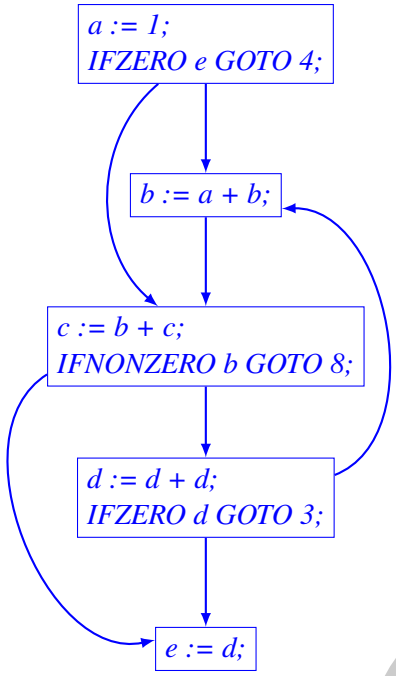
d) Draw the control-flow graph (CFG) of the following program. [5]

```
1: a := 1;
2: IFZERO e GOTO 4;
3: b := a + b;
4: c := b + c;
5: IFNONZERO b GOTO 8;
6: d := d + d;
7: IFZERO d GOTO 3;
8: e := d;
```

[?]

Answer:

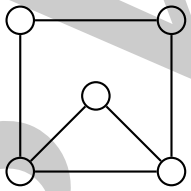
Answers



e) Devise a loop-free CFG whose shape does not correspond to any program in the grammar. [2]

Answer:
Any CFG with fan-out > 2, for instance.

f) Devise a program in this grammar whose *register interference graph* has the following shape. Annotate your program with the set of live variables at the end of each line. [7]



Answer:

```
        {a}
b:=1;   {a,b}
c:=2;   {a,b,c}
d:=a/c; {b,d}
e:=b;   {d,e}
c:=d;   {c,e}
a:=c/e; {a}
```

Answers